

ATPG Fault Detection Using Knowledge Distillation

Automated Test Pattern Generation • Fault Simulation • Machine Learning • Model Compression

Project	ATPG Fault Detection using Knowledge Distillation
Domain	VLSI Design Automation / Design-for-Test
Fault Model	Single stuck-at faults: SA0 / SA1
Implementation	Python-based ATPG, simulation, dataset generation and ML

September 2026

Abstract

This work presents an end-to-end framework for automated test pattern generation (ATPG), stuck-at fault simulation, dataset construction, and knowledge-distillation-based fault detection. A D-algorithm ATPG engine is used to generate test patterns for stuck-at-0 (SA0) and stuck-at-1 (SA1) faults in combinational circuits. The generated patterns are simulated on both fault-free and faulty circuits to obtain labeled examples. A high-capacity teacher classifier is then trained on circuit and fault features. Its probability outputs are transferred to a compact student model using knowledge distillation. In the demonstrated dataset, the teacher achieves 100% accuracy, while the distilled student achieves 75% accuracy with 75% teacher–student prediction agreement. The results demonstrate a complete VLSI-CAD-oriented machine-learning workflow and provide a foundation for future hardware-aware fault diagnosis and efficient ATPG acceleration.

Keywords

ATPG, D-algorithm, stuck-at fault, fault simulation, design-for-testability, knowledge distillation, teacher–student learning, VLSI CAD, machine learning, fault detection

1. Introduction

Testing is a fundamental stage in the digital integrated-circuit design flow. Manufacturing defects and logical faults can cause a circuit to produce incorrect outputs even when its RTL and synthesized structure are functionally correct. Design-for-test (DFT) methodologies therefore generate test patterns that activate faults and propagate their effects to observable outputs.

Traditional ATPG techniques such as the D-algorithm provide a principled method for generating patterns, but their computational cost can increase as circuit complexity and fault count grow. Machine learning can complement classical ATPG by learning relationships between circuit/fault features and fault-detection outcomes. This project investigates that direction while retaining a deterministic ATPG and simulation backend for dataset generation and ground truth.

2. Contributions

- Implementation of a combinational-circuit representation and parser.
- Generation of single stuck-at-0 and stuck-at-1 faults.
- D-algorithm-inspired ATPG procedure for generating test patterns.
- Fault-free versus faulty-circuit simulation and detection labeling.
- Automated dataset generation for machine-learning experiments.
- High-capacity teacher model for fault classification.
- Compact student model trained through knowledge distillation.
- Quantitative evaluation using accuracy, precision, recall, F1 score and teacher–student agreement.
- Result visualizations including confusion matrices, training curves and t-SNE feature plots.

3. System Architecture

The complete workflow is organized into nine stages: circuit parsing, fault generation, ATPG, fault simulation, dataset generation, teacher training, knowledge distillation, student training, and final evaluation.

Stage	Module	Function
1	Circuit Netlist	Parse a combinational circuit/bench representation.
2	Fault Generation	Create SA0 and SA1 fault instances.
3	ATPG Engine	Initialize D-frontier, apply/justify assignments and backtrack.
4	Fault Simulation	Compare fault-free and faulty outputs.
5	Dataset Generation	Store circuit/fault features and detection labels.
6	Teacher Model	Train a high-capacity classifier.
7	Knowledge Distillation	Transfer teacher probability knowledge to the student.
8	Student Model	Train a compact classifier.
9	Final Evaluation	Compute metrics and generate visualizations.

4. Methodology

4.1 Circuit Representation

The circuit is represented as a directed network of logic gates and nets. Primary inputs provide stimulus while gate outputs feed downstream logic and ultimately the primary output. The parser converts the textual circuit description into an internal graph suitable for simulation and ATPG.

4.2 Stuck-at Fault Model

For each selected circuit node, two common single stuck-at faults are considered: SA0, where the node is forced to logic 0, and SA1, where the node is forced to logic 1. A fault is considered detected when a test pattern produces different observable primary-output values for the fault-free and faulty circuits.

4.3 ATPG Using the D-Algorithm

The ATPG stage follows the classical D-algorithm concepts. A fault is first activated by assigning the opposite logic value at the fault site. The resulting discrepancy is represented using D/D-bar values and is propagated through suitable gates toward an observable output. If a conflict is encountered, the search backtracks and tries an alternative assignment.

Conceptually, the ATPG loop is:

1. Initialize the fault and construct the initial D-cube.
2. Propagate the fault effect through the circuit.
3. Justify internal assignments back toward primary inputs.
4. Detect conflicts and backtrack when necessary.
5. Record a primary-input test pattern when the fault effect reaches an output.

4.4 Dataset Generation

Each dataset sample combines circuit/fault descriptors with simulation-derived information. The reported demonstration uses 80 samples and seven input features. Labels represent the fault class or detection outcome used by the classifier.

5. Machine-Learning and Knowledge Distillation

5.1 Teacher Model

The teacher is intentionally higher-capacity and is trained using the generated ATPG dataset. Its role is to learn a high-quality decision function and provide probability distributions that contain more information than hard class labels alone.

5.2 Student Model

The student is a smaller model intended to approximate the teacher with lower computational complexity. This makes the student a candidate for efficient deployment or acceleration-oriented applications.

5.3 Distillation Objective

Knowledge distillation combines the ordinary supervised classification loss with a teacher–student distribution-matching term. A generic formulation used in this project is:

$$L = \alpha L_{CE} + (1 - \alpha) L_{KD}$$

where L_{CE} is the conventional cross-entropy loss and L_{KD} measures the discrepancy between teacher and student soft predictions. The coefficient α controls the balance between hard-label supervision and teacher-derived knowledge.

6. Experimental Setup

Parameter	Value
Dataset size	80 samples
Input features	7
Fault types	SA0 / SA1
Teacher target	High-capacity classifier
Student target	Compact distilled classifier
Primary evaluation	Accuracy, precision, recall, F1, agreement
Visualization	Confusion matrices, training curves, t-SNE

7. Results

The demonstration results show a clear difference between the high-capacity teacher and compact student. The teacher correctly classifies all 80 evaluated samples, whereas the student correctly classifies 60 of the 80 samples.

Metric	Teacher Model	Student Model
Accuracy	100%	75%
Precision	100%	40%
Recall	100%	66.67%
F1 Score	100%	50%
Teacher–Student Agreement	—	75%
Dataset Size	80	80
Features	7	7

7.1 Confusion Matrices

The teacher confusion matrix contains 40 correctly classified SA0 samples and 40 correctly classified SA1 samples, with no observed misclassifications. The student confusion matrix contains 28 correct SA0

predictions, 12 SA0 samples predicted as SA1, 8 SA1 samples predicted as SA0, and 32 correct SA1 predictions.

Model	TN/SA0→SA0	SA0→SA1	SA1→SA0	SA1→SA1
Teacher	SA0→SA0: 40	SA0→SA1: 0	SA1→SA0: 0	SA1→SA1: 40
Student	SA0→SA0: 28	SA0→SA1: 12	SA1→SA0: 8	SA1→SA1: 32

7.2 Training Behaviour

The training visualization shows the teacher loss decreasing during training and the distilled student loss following a comparable decreasing trend. The student does not exactly reproduce the teacher's decision boundary, which is reflected in the lower final classification metrics.

7.3 Feature Visualization

The t-SNE visualization separates the SA0 and SA1 feature representations into distinguishable clusters for both teacher and student representations. This provides a qualitative view of the structure learned from the fault dataset.

8. Discussion

The experiment demonstrates the feasibility of combining classical ATPG with machine learning. The ATPG and fault-simulation stages provide deterministic labels, while the ML stage learns a compact approximation of fault-detection behaviour. Knowledge distillation reduces the student model capacity while retaining a substantial fraction of the teacher's predictive performance.

The reported results should be interpreted as a project demonstration rather than as a statistically general benchmark. The dataset contains only 80 samples and a limited feature space. A larger collection of circuits, fault sites, test patterns, gate types and structural features would be required to establish robustness.

9. Limitations

- The experimental dataset is small.
- The current fault model focuses on single stuck-at faults.
- The benchmark primarily uses combinational-circuit behaviour.
- The reported teacher accuracy is based on the demonstrated dataset and should not be interpreted as generalization accuracy.
- Student-model performance depends strongly on feature engineering, architecture and distillation hyperparameters.
- The current implementation does not yet provide hardware synthesis or measured silicon/FPGA deployment cost.

10. Future Work

- Scale the dataset across ISCAS, ITC and other benchmark circuits where licensing and reproducibility permit.
- Add transition, bridging and delay-oriented fault models.
- Use circuit-graph representations and graph neural networks.
- Explore ATPG-guided active learning to select the most informative training samples.

- Compare knowledge distillation with pruning, quantization and other model-compression methods.
- Measure inference latency, memory footprint and energy on CPU, FPGA or ASIC-oriented targets.
- Integrate fault collapsing and structural circuit features into the learning pipeline.
- Investigate ML-assisted ATPG search and reinforcement-learning-based pattern generation.

11. Reproducibility and Repository Structure

The project is organized as a Git repository containing the ATPG engine, circuit parser, simulator, fault generation, dataset generation, teacher/student models, evaluation scripts, examples, models and result files.

```
atpg-fault-detection-distillation/
├── atpg.py
├── circuit.py
├── parser.py
├── simulator.py
├── fault.py
├── fault_collapsing.py
├── d_algorithm.py
├── dataset_generator.py
├── teacher_model.py
├── student_model.py
├── student_evaluation.py
├── final_demo.py
├── main.py
├── dataset/
├── models/
├── examples/
├── tests/
├── docs/
├── requirements.txt
├── methodology.md
├── results.md
└── README.md
```

12. Conclusion

This project establishes an end-to-end VLSI-CAD workflow in which deterministic ATPG and fault simulation generate labeled data for machine-learning-based fault detection. A high-capacity teacher model provides soft knowledge to a compact student through knowledge distillation. On the demonstrated 80-sample dataset, the teacher reaches 100% accuracy and the student reaches 75% accuracy with 75% prediction agreement. The framework therefore serves as a practical foundation for further research into hardware-aware machine learning, efficient fault diagnosis and ML-assisted design-for-test methodologies.

References

6. P. Goel, "An Implicit Enumeration Algorithm to Generate Tests for Combinational Logic Circuits," IEEE Transactions on Computers.
7. J. P. Roth, "Diagnosis of Automata Failures: A Calculus and a Method," IBM Journal of Research and Development.

8. L. W. M. H. Lim and others, foundational literature on automated test pattern generation and digital circuit testing.
9. G. H. De Micheli, Synthesis and Optimization of Digital Circuits, McGraw-Hill.
10. G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," NIPS Deep Learning Workshop.
11. F. Chollet, Deep Learning with Python, Manning Publications.